PinionAI Agent Runners - GCP, Streamlit Cloud, Slack, Teams and CLI Examples

This project allows you to build Agentic AI application using the Streamlit framework, or communicate with it via Slack, Teams or a local terminal CLI, and the PinionAI python library to serve Agents you author in PinionAI studio.

Instructions below are included for deploying this application to Cloud Run or Streamlit Community Cloud. You can run the application locally, configure it to be accessible in Slack or Teams, as well as an online or local Streamlit chat interface.

If desired, you can also iframe the project once deployed inside your current infrastructure.
See: iframing PinionAI

For complete documentation please see: Full Documentation

For questions or support please contact us at: Contact Form

Local Deployment - Locally Running your AI Agent

To run the Streamlit Application locally (on cloud shell), we need to perform the following steps:

1. Decisions, decisions... By default, the application will install `pinionai` from PyPI. First thing to know is that pinionai will include a number of dependencies, but also can also install optional dependencies. Depending on what you plan to use in your agents, you should consider which optional libraries you might want to install.

If desired, edit the `pyproject.toml`, before generating the requirements.txt and pip install. By default, `pinionai[all]` is installed, but there are additional options or changes you can make.

PinionAI Python Library

This is the official Python client library for the PinionAI platform. It provides a convenient, asynchronous way to interact with PinionAI agents, manage sessions, and use its various features including AI interactions and gRPC messaging.

PinionAI Python Package: PyPi Python

Installation

From PyPI

This package is available on PyPI and can be installed with `pip` or `uv`. We recommend `uv` for its speed.

With `uv`

If you don't have `uv`, you can install it from astral.sh.

```
# On macOS and Linux
curl -LsSf https://astral.sh/uv/install.sh | sh
#OR
brew install uv
```

```
# On Windows
powershell -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Once `uv` is installed, you can install the `pinionai` package from PyPI:

```
uv pip install pinionai
```

With `pip`

If you prefer to use pip, you can still install the package with:

```
pip install pinionai
```

Optional Features

The base install covers the common client and session workflow. Optional integrations are grouped into extras so you can keep the install lean and only pull in the packages you need.

- gcp: Google Cloud Storage support (google-cloud-storage)
- aws: AWS S3 support (boto3)
- openai: Support for OpenAI models (openai)
- anthropic: Support for Anthropic models (anthropic)
- javascript: Support for running JavaScript snippets (mini-racer)
- sendgrid: Support for running SendGrid delivery
- twilio: Support for SMS delivery
- mcp: FastMCP tool integration (fastmcp)
- data: Tabular and CSV helpers (pandas)
- postgres: PostgreSQL connector support (psycopg)
- google: Google Gemini / Vertex AI support (google-genai)

To install one or more optional features, specify them in brackets. For example, to get support for GCP, AWS, and Google AI:

```
pip install "pinionai[gcp,aws,google]"
```

To install all optional features at once, use the `all` extra:

```
pip install "pinionai[all]"
```

Options include:

- dev = ["build", "twine", "ruff", "grpcio-tools",]
- gcp = ["google-cloud-storage"]
- aws = ["boto3"]
- openai = ["openai"]
- anthropic = ["anthropic"]
- javascript = ["mini-racer"]
- sendgrid = ["sendgrid"]
- twilio = ["twilio"]
- data = ["pandas"]
- google = ["google-genai>=1.72.0"]
- all = ["pinionai[gcp,aws,openai,anthropic,javascript,twilio,sendgrid,data,google]"]

2. Setup the Python virtual environment and install the dependencies:

   In Cloud Shell, execute the following commands:

For a new venv:

```
uv venv pinionai-streamlit # 1. Create a new virtual environment with uv
source pinionai-streamlit/bin/activate # 2. Activate the venv

# 3. Compile your dependencies from pyproject.toml into a lockfile.
#    This reads requirements.in and creates/updates requirements.txt.
uv pip compile requirements.in -o requirements.txt
# or
uv pip compile --upgrade requirements.in -o requirements.txt

# 4. Install the locked dependencies into your venv.
uv pip sync requirements.txt
```

> [!TIP] uv is a modern, extremely fast Python package installer from the creators of ruff. If you don't have it, you can install it with pip install uv or by following the official installation guide. Using uv can significantly speed up environment creation and dependency installation.

3. Your application requires access to a few environment variables, see the .env.example file. Use this dotenv file as a template. Copy it to `.env` and fill in your actual credentials, and do not commit it to version control.

   - client_id = '<YOUR_CLIENT_ID_HERE>'
   - client_secret = '<YOUR_CLIENT_SECRET_HERE>'
   - agent_id = '<YOUR_AGENT_ID_HERE>'
   - host_url = 'https://microservice-72loomfx5q-uc.a.run.app'

`client_id` and `client_secret` are found within your PinionAI Studio application. You can generate a new client secret in the PinionAI Studio portal if desired. `agent_id` is the unique identifier of the AI agent you create in PinionAI Studio and want to serve. `host_url` is the location of PinionAI API server.

NOTE: AI Agent LLM keys, GCP_PROJECT, WEBSOCKET_URL and many other variables are set within PinionAI Studio itself. You will need to configure these to run your agent.

NOTE: Do not commit your `.env` file to version control if it contains sensitive information. Add `.env` to your `.gitignore`.

4. To run the application locally, execute the following commands:

*Key for google ADC to run gemini LLM locally.

```
gcloud auth application-default login
```

```
streamlit run chat.py
```

The application will startup and you will be provided a local URL to the application.

CLI Deployment - Running the local terminal (CLI) chat client: `chat_cli.py`

A lightweight terminal-based client is included for environments where Streamlit isn't available or when you prefer a CLI workflow. The CLI mirrors the same agent logic used by the Streamlit app but interacts via stdin/stdout.

Prerequisites are the same as for the Streamlit app: a Python environment with the dependencies installed and required environment variables set (or a `.env` file present).

Create / activate your virtualenv (example using `uv`):

```
uv venv pinionai-streamlit
source pinionai-streamlit/bin/activate
uv pip sync requirements.txt
```

Set the same environment variables you use for the Streamlit app (or copy `.env.example` to `.env` and edit):

```
client_id=<YOUR_CLIENT_ID_HERE>
client_secret=<YOUR_CLIENT_SECRET_HERE>
agent_id=<YOUR_AGENT_ID_HERE>
host_url=https://microservice-72loomfx5q-uc.a.run.app
```

Run the CLI client:

```
python chat_cli.py
```

Slack Deployment - Running the Slack bot: `chat_slack.py`

A Slack-based client is included to allow interacting with your PinionAI agents directly from a Slack channel. It uses `slack_bolt` with Socket Mode for easy setup without needing a public endpoint.

Slack Integration Document

For Slack Marketplace / App Directory submission, use the new guides:

- Slack Marketplace Distribution Guide
- Slack Marketplace Submission Checklist
- Slack Marketplace Deployment on GCP
- Slack Marketplace Deployment on DigitalOcean

> The Socket Mode setup above is appropriate for private or internal Slack bots. Marketplace submission requires an HTTP-based public deployment with public distribution enabled.

Prerequisites

1. **Create a Slack App:** Go to api.slack.com/apps and create a new app.
2. **Tokens:**
   - **Bot User OAuth Token (`SLACK_BOT_TOKEN`)**: Found under "OAuth & Permissions". Needs `chat:write` and `files:read` scopes.
   - **App-Level Token (`SLACK_APP_TOKEN`)**: Found under "Basic Information". Needs `connections:write` scope and must be created with the `socket_mode` scope.
3. **Socket Mode:** Enable Socket Mode in your Slack app settings.
4. **Event Subscriptions:** Enable Events and subscribe to `message.channels` (and `message.groups` if needed).

Setup and Running

Install dependencies (ensure `slack-bolt` is installed):

```
uv pip compile requirements.in -o requirements.txt
# or
uv pip compile --upgrade requirements.in -o requirements.txt
uv pip sync requirements.txt
```

Set environment variables in your `.env` file:

```
SLACK_BOT_TOKEN=xoxb-...
SLACK_APP_TOKEN=xapp-...
```

```
host_url=https://api.pinionai.com
# Optional: default agent
client_id=<YOUR_CLIENT_ID_HERE>
client_secret=<YOUR_CLIENT_SECRET_HERE>
agent_id=<YOUR_AGENT_ID_HERE>
```

Run the Slack bot locally:

```
python chat_slack.py
```

Production deployment on a VM

For production, use the Slack-specific container image defined in `Dockerfile.slack`.

```
docker build -f Dockerfile.slack -t pinionai-slack:latest .
```

Then run it with a server-side `.env` file that contains the same values you would normally place in `deploy/prod-slack/env.yaml`.

```
docker run -d \
  --name pinionai-slack \
  --restart unless-stopped \
  --env-file /path/to/.env \
  pinionai-slack:latest
```

This approach works well on both Google Compute Engine and DigitalOcean droplets because the bot needs a long-running process rather than a scale-to-zero web workload.

Features

- **Message Interaction:** Type in the channel to talk to the agent.
- **AIA File Loading:** Upload a `.aia` file to the channel. The bot will load that agent for all subsequent messages in that channel.
- **Private AIA Files:** If an AIA file is encrypted, the bot will ask for the `key_secret` in the next message.
- **/end:** Use the `/end` command (as a regular message) to clear the current agent session.
- **!end** Alternatively, if you do not want to configure a Slack command for `/end`, you can simply type `!end` as a regular message to clear your session.

Microsoft Teams Deployment - Running the Teams bot: `chat_teams.py`

A Microsoft Teams-based client is included to allow interacting with your PinionAI agents from Teams. It uses the Microsoft Bot Framework SDK.

Teams Integration Document

Prerequisites

1. **Register a Bot:** Register a bot with the Microsoft Bot Framework or via the Azure Portal (Azure Bot service).
2. **App ID and Password:** Obtain your Microsoft App ID and App Password.
3. **Messaging Endpoint:** Your bot will need a public HTTPS endpoint (e.g., using `ngrok` for local testing) pointing to `/api/messages`.

Setup and Running

Install dependencies:

```
uv pip compile requirements.in -o requirements.txt
# or
uv pip compile --upgrade requirements.in -o requirements.txt
uv pip sync requirements.txt
```

Set environment variables in your `.env` file:

```
TEAMS_APP_ID=your-app-id
TEAMS_APP_PASSWORD=your-app-password
host_url=https://api.pinionai.com
```

Run the Teams bot:

```
python chat_teams.py
```

Features

- **Message Interaction:** Talk to the agent in Teams.
- **AIA File Loading:** Upload a `.aia` file as an attachment.
- **Private AIA Files:** Handles encrypted agents via secret key prompt.
- **/end:** Reset the session.

Loading an Agent from an AIA File

The CLI supports loading agents from `.aia` (AI Agent) files. This is useful when you don't have environment variables configured or want to run a specific agent file directly.

Option 1: Pass the AIA file via command-line flag

```
python chat_cli.py -f path/to/agent.aia
```

Or use the long form:

```
python chat_cli.py --aia-file path/to/agent.aia
```

Option 2: Interactive AIA file loading

If environment variables (`client_id`, `client_secret`, `agent_id`) are not set or the client initialization fails, the CLI will prompt you to provide the path to an `.aia` file:

```
Failed to initialize PinionAI client from environment: ...
Enter path to .aia file to load the agent (or leave empty to abort):
```

Encrypted AIA Files (Key Secret)

If your `.aia` file is encrypted and requires a `key_secret`:

1. The CLI will detect this and prompt: `This AIA file requires a key_secret to decrypt.`
2. Enter your key_secret securely (input is hidden):

   ```
   Enter key_secret:
   ```

3. The CLI will attempt to decrypt and load the agent. You have up to 3 attempts.

Basic interactive commands inside the CLI:

- Type your message and press Enter to send it to the agent.
- /end — end the chat session and exit the client.
- /continue — force a short poll for updates and display any new messages.

Notes:

- The CLI prints simple text output (no avatars or rich markdown rendering).
- Live-agent transfers (gRPC) are supported if the agent requests a transfer; the CLI will attempt to start the gRPC listener and poll for agent responses.
- If you rely on Streamlit-specific session-state features, the CLI may behave slightly differently; the core message flow and client API usage remain the same.
- The same flexibility as the Streamlit app applies: use environment variables for default agent configuration, or supply an `.aia` file to run any agent on demand.

Streamlit Community Cloud Deployment - Production Deploy AI Agent to `Streamlit Community Cloud`

This guide provides step-by-step instructions on how to deploy the Pinion AI chat application to Streamlit Community Cloud, including configuration of required environment variables using a `.env` file, or `env.yaml`.

Steamlit Deployment Document

Why Streamlit Community Cloud:

Streamlit Community Cloud is a free, web-based platform for building and sharing data apps created with the Streamlit framework. It allows users to easily deploy their apps from GitHub repositories and share them with a public or private audience with minimal configuration. The platform handles containerization and provides a simple interface for managing apps, and it can be integrated with tools like GitHub Codespaces for browser-based development.

Key Features

- **Free deployment**: It is a free service for sharing Streamlit apps, with a free resource limit of 1GB.
- **Fast deployment**: Apps can be deployed in minutes with just a few clicks by connecting a GitHub repository.
- **App sharing**: Users can share their apps publicly with the world or with specific individuals through private sharing.
- **GitHub integration**: It connects to GitHub repositories for version control and deployment.
- **App management**: It provides a dashboard for managing apps, accessing logs, and updating them.
- **Browser-based development**: It integrates with GitHub Codespaces to allow users to write and edit code directly in their browser.

Deployment Process:

This is the most straightforward method for quick deployments, especially for demos or applications without strict enterprise requirements.

1. **Copy PinionAI Agent Github**: Copy Example PinionAI Agent Github: A Streamlit PinionAI Github project, ready for use with PinionAI Studio and Streamlit Cloud.
2. **Your Public GitHub**: Host your Streamlit app files (including chat.py and requirements.txt) in a public GitHub repository.
3. **Sign Up for Streamlit Community Cloud**: Using your GitHub account, Sign up for Streamlit Community Cloud
4. **Connect Repository** Connect your GitHub repository and specify the main app file (e.g.,chat.py).

- Fill out form
- Click `Advanced` and Configure any necessary environment variables in TOML format.

```
client_id = "<YOUR_CLIENT_ID_HERE>"
client_secret = "<YOUR_CLIENT_SECRET_HERE>"
agent_id = "<YOUR_AGENT_ID_HERE>"
host_url = "https://api.pinionai.com"
'''
```

OR

- For a generic app that allows any agent to be loaded via .AIA file (`ai agent`) file, you do not need client_id, client_secret or a specific agent_id.

```
host_url = "https://api.pinionai.com"
'''
```

5. Deploy the app.

- **Advantages**: Free, easy to set up, good for sharing and testing.
- **Disadvantages:** Limited control over infrastructure, not ideal for highly sensitive data or strict compliance needs.

Service Account Permissions:

Gemini/Vertex AI Connectors (for non-GCP hosted agents)

```
**Note:** To allow PinionAI agents to use Gemini/Vertex AI when not hosted
in GCP, you must add a service account with the appropriate permissions to
the Agent configuration in PinionAI Studio (in Connectors Tab). A service
account is a special type of Google account intended to represent a non-
human user (like an application) that needs to authenticate and be
authorized to access data in Google APIs. Configure connectors with the
following special grant types:

    **For Google Gemini/Vertex AI Access (non-GCP hosted):**
    *   `connector_grant_type`: Set this to `gcs_service_account`.
    *   `connector_client_id`: Can be left blank or used for the GCP
Project ID.
    *   `connector_client_secret`: Paste the **entire JSON content** of
your GCP service account key file here.

    **Note on GCP Permissions:** Ensure that the service account associated
with the provided key has the necessary permissions you desire. Use
Gemini/Vertex AI User (`roles/aiplatform.user`) for example.
```

Steps to Create a Google Service Account and Download Key File

```
1. Go to the [Google Cloud Console](https://console.cloud.google.com/).
2. Select your project or create a new one.
3. Navigate to "IAM & Admin" > "Service Accounts".
4. Click "Create Service Account".
5. Fill in the service account details and click "Create".
6. Grant the service account the necessary roles (e.g., Vertex AI User).
7. Click "Done" to finish creating the service account.
8. Find your service account in the list and click on it.
```

```
9. Go to the "Keys" tab and click "Add Key" > "JSON".
10. A JSON key file will be downloaded to your computer. This file contains
the credentials needed to authenticate as the service account.
11. Use the contents of this file in the `Client Secret` field of your
connector in Pinion AI Studio configuration. Save and select the connector
in your desired Agent.
```

GCP Deployment - Production Deploy AI Agent to `Google Cloud Run`

This guide provides step-by-step instructions on how to deploy the Pinion AI chat application to Google Cloud Run, including configuration of required environment variables using a `.env` file.

GCP Deployment Document

Prerequisites

Before you begin, ensure you have the following installed and configured:

1. **Google Cloud SDK (`gcloud` CLI):** Google Cloud CLI is installed and configured.
2. **Docker:** Docker is installed.
3. A Google Cloud Project. If you don't have one, create one in the Google Cloud Console.

Log in to your Google Cloud account and set your project.

```
gcloud auth login
gcloud config set project YOUR_PROJECT_ID
```

Replace `YOUR_PROJECT_ID` with your actual Google Cloud project ID.

Step 1: Export out a few core variables

```
# Set basic environment variables for your project and desired service
account name
export PROJECT_ID=$(gcloud config get-value project)
export SERVICE_ACCOUNT_NAME="pinionai-client-runner"
```

Step 2: Create a Dedicated Service Account and Grant Necessary IAM Roles (only do once)

- NOTE: only do step 2 once

First, create a new, dedicated service account for your Cloud Run service. This ensures the service has only the permissions it needs, following the principle of least privilege.

```
# Create the service account
gcloud iam service-accounts create "${SERVICE_ACCOUNT_NAME}" \
```

```
    --display-name="Cloud Run PinionAI Client Service Account" \
    --project="${PROJECT_ID}"
```

Next, grant the required IAM roles to the new service account. These permissions allow the service to interact with Vertex AI, Cloud Storage, and use the enabled APIs.

```
# Get the full email address of the service account
export
SERVICE_ACCOUNT_EMAIL="${SERVICE_ACCOUNT_NAME}@${PROJECT_ID}.iam.gserviceac
count.com"

# To execute Gemini prompts and use RAG Stores
gcloud projects add-iam-policy-binding "${PROJECT_ID}" \
    --member="serviceAccount:${SERVICE_ACCOUNT_EMAIL}" \
    --role="roles/aiplatform.user"

# Grant the Storage Object Admin role for uploading files to GCS
gcloud projects add-iam-policy-binding "${PROJECT_ID}" \
    --member="serviceAccount:${SERVICE_ACCOUNT_EMAIL}" \
    --role="roles/storage.objectAdmin"

# Grant the Service Usage Consumer role to allow using the APIs
gcloud projects add-iam-policy-binding "${PROJECT_ID}" \
    --member="serviceAccount:${SERVICE_ACCOUNT_EMAIL}" \
    --role="roles/serviceusage.serviceUsageConsumer"

# Grant Cloud Run invoker role
gcloud projects add-iam-policy-binding "${PROJECT_ID}" \
    --member="serviceAccount:${SERVICE_ACCOUNT_EMAIL}" \
    --role="roles/run.invoker"
```

Deployment Steps

1. Set Up Your Environment

Define some environment variables to make the following commands easier to manage. Choose a region that supports Cloud Run.

```
export PROJECT_ID=$(gcloud config get-value project)
export REGION="us-central1" # Or your preferred region
export REPOSITORY="pinionai-chat-repo"
export IMAGE_NAME="pinionai-chat"
export SERVICE_ACCOUNT_NAME="pinionai-client-runner"
export
SERVICE_ACCOUNT_EMAIL="${SERVICE_ACCOUNT_NAME}@${PROJECT_ID}.iam.gserviceac
count.com"
```

2. Enable Required APIs (only need to do this once)

You need to enable the Artifact Registry API (to store your Docker image) and the Cloud Run API (to run your service).

```
gcloud services enable artifactregistry.googleapis.com run.googleapis.com
```

3. Create an Artifact Registry Repository (only need to do this once)

Create a Docker repository in Artifact Registry to host your container image.

```
gcloud artifacts repositories create ${REPOSITORY} \
    --repository-format=docker \
    --location=${REGION} \
    --description="Docker repository for the PINION AI Chat application"
```

4. Configure Docker Authentication

Configure the Docker command-line tool to authenticate with Artifact Registry.

```
gcloud auth configure-docker ${REGION}-docker.pkg.dev
```

5. Build and Push the Docker Image

Build the Docker image from the `Dockerfile` in the project root, tag it, and push it to your Artifact Registry repository.

```
# Define the full image URI
export IMAGE_URI="${REGION}-
docker.pkg.dev/${PROJECT_ID}/${REPOSITORY}/${IMAGE_NAME}:latest"

# As a deploy (single command to build and push)
gcloud builds submit --tag ${IMAGE_URI}

# --- OR ----
# Build the Docker image from the project root
docker build -t ${IMAGE_URI} .

# Push the image to Artifact Registry
docker push ${IMAGE_URI}
```

6. Deploy to Google Cloud Run with Environment Variables

Deploy your container image to Cloud Run and pass the environment variables from your `.env` file. You can specify each variable using the `--set-env-vars` flag, Or use the `--env-vars-file` flag with a YAML file.

Use the `--service-account` flag to specify the identity your service will run as.

Option 1: Using `--set-env-vars`

Extract the variables from your `.env` file and pass them as a comma-separated list:

```
gcloud run deploy ${IMAGE_NAME} \
    --image=${IMAGE_URI} \
    --port=8080 \
    --service-account=${SERVICE_ACCOUNT_EMAIL} \
    --project=${PROJECT_ID} \
    --region=${REGION} \
    --platform=managed \
    --allow-unauthenticated \
    --min-instances 0 \
    --cpu-boost \
    --no-cpu-throttling \
    --session-affinity \
    --timeout=3600 \
    --set-env-vars
client_id=YOUR_CLIENT_ID_HERE,client_secret=YOUR_CLIENT_SECRET_HERE,etc...
```

- environment variables needed are:

```
client_id = '<YOUR_CLIENT_ID_HERE>'
client_secret = '<YOUR_CLIENT_SECRET_HERE>'
agent_id = '<YOUR_AGENT_ID_HERE>'
host_url = 'https://microservice-72loomfx5q-uc.a.run.app' #
'<PINIONAI_API_HOST_URL_HERE>'
version = ''
```

Important Note: When hosting an Agent you can serve a specific agent configuration, For a generic app that allows any agent to be loaded via .AIA file (`ai agent`) file, you do not need `client_id`, `client_secret` or a specific `agent_id`. `client_id`, `client_secret` and `agent_id` are **NOT Required** variables.

OR

```
host_url = 'https://microservice-72loomfx5q-uc.a.run.app'
```

Option 2: Using an Environment Variables YAML File

Convert your `.env` file to a YAML file (e.g., `env.yaml`) and save:

```
client_id: <YOUR_CLIENT_ID_HERE>
client_secret: <YOUR_CLIENT_SECRET_HERE>
agent_id: <YOUR_AGENT_ID_HERE>
host_url: https://api.pinionai.com
version: "" # specific version include: draft, development, test, live,
archived
```

Important Note: When hosting an Agent you can serve a specific agent configuration, For a generic app that allows any agent to be loaded via .AIA file (`ai agent`) file, you do not need `client_id`, `client_secret` or a specific `agent_id`. `client_id`, `client_secret` and `agent_id` are **NOT Required** variables.

Then deploy with:

```
gcloud run deploy ${IMAGE_NAME} \
    --image=${IMAGE_URI} \
    --port=8080 \
    --service-account=${SERVICE_ACCOUNT_EMAIL} \
    --project=${PROJECT_ID} \
    --region=${REGION} \
    --platform=managed \
    --allow-unauthenticated \
    --min-instances 0 \
    --cpu-boost \
    --no-cpu-throttling \
    --session-affinity \
    --timeout=3600 \
    --env-vars-file deploy/prod-agent/env.yaml
```

OR

```
--env-vars-file deploy/prod-aia-file/env.yaml
```

Critical Cloud Run Performance Flags Used Here:

- --no-cpu-throttling: This turns on the "CPU Always Allocated" feature. This is absolutely vital for your app because it prevents Cloud Run from freezing your asynchronous gRPC background polling loops (listen_to_grpc_updates) between active user keypresses.
- --session-affinity: Each chat's PinionAI client lives in the memory of the instance that created it, so a browser must keep reaching the same instance. Do not add Redis (`REDIS_URL`) to "fix" this: the client cannot be serialized, so it would be lost after every event.
- --timeout=3600: Increases the request timeout to 1 hour. This ensures long-lived WebSocket chat connections stay alive and don't aggressively drop out if the user takes a brief pause during a chat session.

- --min-instances: Allowing for miniimum instances to spin down to 0, saves on cost while developing, but is not recommended for production. It will increase cold starts and spin up time substantially, and not recommended for production.

Important Note: When hosting an Agent you can serve a specific agent configuration, For a generic app that allows any agent to be loaded via .AIA file (`ai agent`) file, you do not need `client_id`, `client_secret` or a specific `agent_id`. `client_id`, `client_secret` and `agent_id` are **NOT Required** variables.

During the deployment process, `gcloud` will prompt you to confirm the settings. Once you confirm, it will deploy the service and provide you with a **Service URL**.

7. Access Your Application

Once the deployment is complete, you can access your Streamlit application by navigating to the **Service URL** provided in the command-line output. If you want to use a domain name, you will need to create a CNAME with your domain provider that points to this Service URL. However at this time, Google does not recommend this for production services.

- Cloud Run domain mappings are in the preview launch stage. Due to latency issues, they are not production-ready and are not supported at General Availability. At the moment, this option is not recommended for production services.

8. Maintaining 'Always Ready' Service

To keep a Cloud Run server running without it shutting down due to inactivity, you must set a minimum number of instances to maintain a warm, ready state even when there's no traffic, or use manual scaling to keep a specific number of instances running all the time. You should also ensure your application is configured for instance-based billing to support potential background activities and not rely on scaling to zero.

1. **Configure minimum instances.**

Change the gcloud run deploy ${IMAGE_NAME} command above, and include --cpu-boost \ flag when deploying

OR

```
gcloud run services update SERVICE_NAME --min-instances=1
```

OR

- Navigate to your Cloud Run service: in the Google Cloud console.
- Edit the service.
- Adjust the "Minimum number of instances" setting: to a value greater than zero. This tells Cloud Run to keep at least that many instances running, even when they are idle.
- Save the changes: to deploy the updated configuration.

2. **USE CPU boost**

You can use CPU boost tag to double CPU effort for the first 10 seconds. This helps speed the application startup when running a cold start.

Change the gcloud run deploy ${IMAGE_NAME} command above, and set the flag --min-instances 1 \ when deploying

3. **Consider CPU always allocated** Additionally, this feature ensures that a container instance's CPU is fully available for background processing, not just during request handling. It is typically used in combination with minimum instances.

- In Google Cloud Console: In the service editor, navigate to the Container, networking, security section. In the Container tab, select CPU is always allocated.
- In gcloud CLI:

```
gcloud run services update SERVICE_NAME --cpu-throttling --cpu-
throttling=never
```

Installing Development Pinionai

By default, the application will install `pinionai` from PyPI. If you want to install the latest development version from GitHub, comment out the `pinionai` line in `requirements.in` (or `requirements.txt`) and uncomment the GitHub line.

Generic App Deployment cheatsheet - AIA Files

```
export PROJECT_ID=$(gcloud config get-value project)
export SERVICE_ACCOUNT_NAME="pinionai-client-runner"
export
SERVICE_ACCOUNT_EMAIL="${SERVICE_ACCOUNT_NAME}@${PROJECT_ID}.iam.gserviceac
count.com"
export REGION="us-central1" # Or your preferred region
export REPOSITORY="pinionai-chat-files"
export IMAGE_NAME="pinionai-chat-files"
export IMAGE_URI="${REGION}-
docker.pkg.dev/${PROJECT_ID}/${REPOSITORY}/${IMAGE_NAME}:latest"
```

Only once here

```
gcloud artifacts repositories create ${REPOSITORY} \
    --repository-format=docker \
    --location=${REGION} \
    --description="Docker repository for the PINION AI Chat AIA File
application"
```

```
gcloud builds submit --tag ${IMAGE_URI}
```

```
gcloud run deploy ${IMAGE_NAME} \
    --image=${IMAGE_URI} \
    --port=8080 \
    --service-account=${SERVICE_ACCOUNT_EMAIL} \
    --project=${PROJECT_ID} \
    --region=${REGION} \
    --platform=managed \
    --allow-unauthenticated \
    --min-instances 0 \
    --cpu-boost \
    --no-cpu-throttling \
    --session-affinity \
    --timeout=3600 \
    --env-vars-file deploy/prod-aia-file/env.yaml
```

Generic App Deployment cheatsheet - Cloud Run for Slack

!!! Change Dockerfile to launch the chat_slack.py file. Change chat.py to chat_slack.py if you want to run the Slack version of the app CMD ["streamlit", "run", "chat_slack.py", "--server.port=8080", "--server.address=0.0.0.0", "--server.headless=true"]

```
export PROJECT_ID=$(gcloud config get-value project)
export SERVICE_ACCOUNT_NAME="pinionai-client-runner"
export
SERVICE_ACCOUNT_EMAIL="${SERVICE_ACCOUNT_NAME}@${PROJECT_ID}.iam.gserviceac
count.com"
export REGION="us-central1" # Or your preferred region
export REPOSITORY="pinionai-chat-slack"
export IMAGE_NAME="pinionai-chat-slack"
export IMAGE_URI="${REGION}-
docker.pkg.dev/${PROJECT_ID}/${REPOSITORY}/${IMAGE_NAME}:latest"
```

Only once here

```
gcloud artifacts repositories create ${REPOSITORY} \
    --repository-format=docker \
    --location=${REGION} \
    --description="Docker repository for the PINION AI Slack application"
```

```
gcloud builds submit --tag ${IMAGE_URI}
```

```
gcloud run deploy ${IMAGE_NAME} \
    --image=${IMAGE_URI} \
    --port=8080 \
    --service-account=${SERVICE_ACCOUNT_EMAIL} \
    --project=${PROJECT_ID} \
    --region=${REGION} \
    --platform=managed \
    --allow-unauthenticated \
    --min-instances 0 \
    --cpu-boost \
    --env-vars-file deploy/prod-slack/env.yaml
```